

프로그래밍응용개념

#04 조건문과 반복문

국립한밭대학교 인공지능 소프트웨어학과

이성민

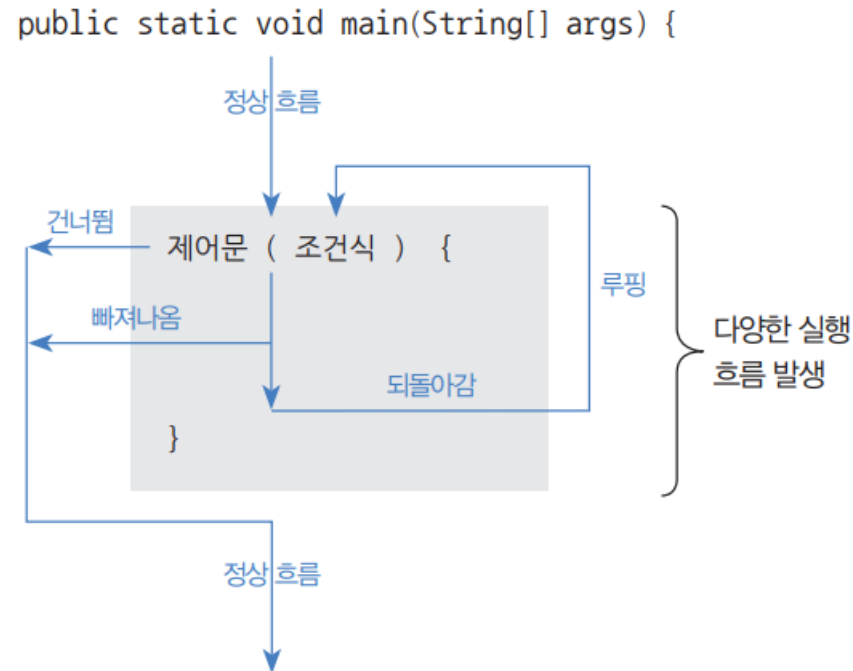
제어문

- 조건문
- 반복문

코드 실행 흐름 제어

- 코드 실행

- 실행 흐름: main() 메소드의 시작에서 끝까지
- 제어문: 실행 흐름을 개발자가 원하는 방향으로 바꿀 수 있도록 해줌
- 루핑: 반복문이 실행 완료된 경우 제어문 처음으로 다시 되돌아 감



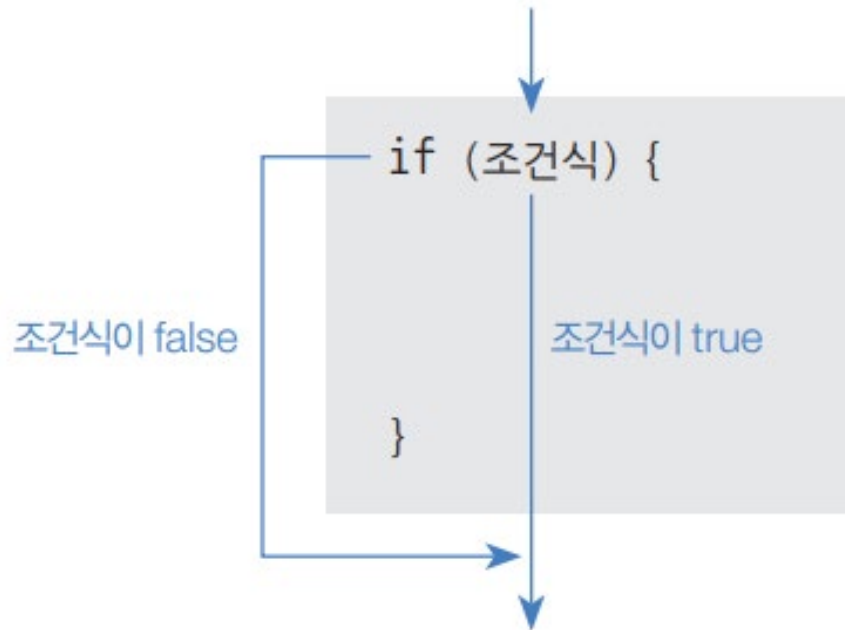
코드 실행 흐름 제어

- 제어문 종류
 - 조건문과 반복문

조건문	반복문
if 문, switch 문	for 문, while 문, do-while 문

코드 실행 흐름 제어

- if 문
 - 조건식의 결과에 따라 블록 실행 여부가 결정
 - 조건식에는 true 또는 false 값을 산출할 수 있는 연산식이나 boolean 변수를 받음
 - 조건식이 true이면 블록을 실행하고 false이면 블록을 실행하지 않음



코드 실행 흐름 제어

- if 문

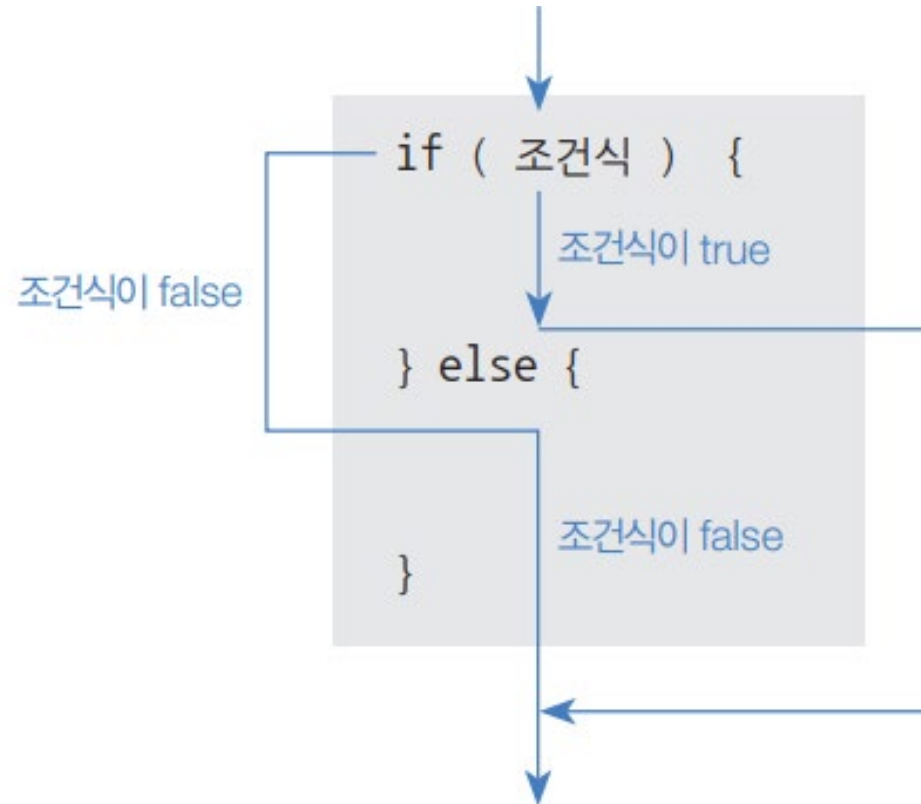
```
int score = 93;

if(score >= 90) {
    System.out.println("점수가 90보다 큽니다.");
    System.out.println("등급은 A 입니다.");
}

if(score < 90) {
    System.out.println("점수가 90보다 작습니다.");
    System.out.println("등급은 B 입니다.");
}
```

코드 실행 흐름 제어

- if-else 문
 - 조건식이 true이면 if 문 블록이 실행되고, false이면 else 블록이 실행



코드 실행 흐름 제어

- if-else 문

```
int score = 93;

if(score >= 90) {
    System.out.println("점수가 90보다 큽니다.");
    System.out.println("등급은 A 입니다.");
}

if(score < 90) {
    System.out.println("점수가 90보다 작습니다.");
    System.out.println("등급은 B 입니다.");
}
```



```
int score = 85;

if(score >= 90) {
    System.out.println("점수가 90보다 큽니다.");
    System.out.println("등급은 A 입니다.");
} else {
    System.out.println("점수가 90보다 작습니다.");
    System.out.println("등급은 B 입니다.");
}
```


코드 실행 흐름 제어

- if-else 문

```
int score = 75;

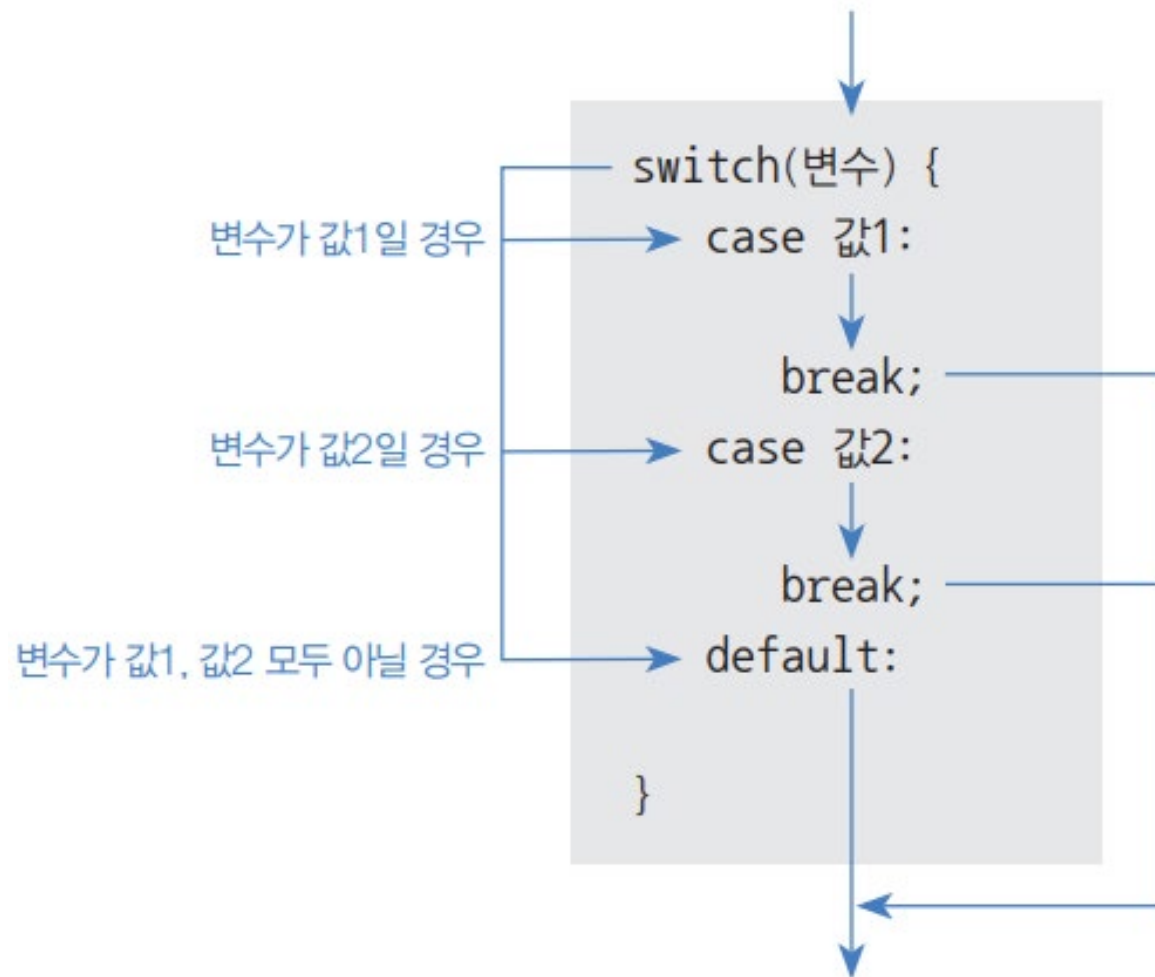
if(score >= 90) {
    System.out.println("점수가 100~90 입니다.");
    System.out.println("등급은 A 입니다.");
} else if(score >= 80) {
    System.out.println("점수가 80~89 입니다.");
    System.out.println("등급은 B 입니다.");
} else if(score >= 70) {
    System.out.println("점수가 70~79 입니다.");
    System.out.println("등급은 C 입니다.");
} else {
    System.out.println("점수가 70 미만 입니다.");
    System.out.println("등급은 D 입니다.");
}
```

코드 실행 흐름 제어

- switch 문
 - 괄호 안의 변수값에 따라 해당 case로 가서 실행문을 실행.
 - 변수값과 동일한 값을 갖는 case가 없으면 default로 가서 실행문을 실행하며, default 생략 가능
 - break는 다음 case를 실행하지 않고 switch 문을 빠져나갈 때 사용
 - break가 없다면 다음 case가 연달아 실행

코드 실행 흐름 제어

- switch 문



코드 실행 흐름 제어

- switch 문

```
int num = (int)(Math.random()*6) + 1;

switch(num) {
    case 1:
        System.out.println("1번이 나왔습니다.");
        break;
    case 2:
        System.out.println("2번이 나왔습니다.");
        break;
    case 3:
        System.out.println("3번이 나왔습니다.");
        break;
    case 4:
        System.out.println("4번이 나왔습니다.");
        break;
    case 5:
        System.out.println("5번이 나왔습니다.");
        break;
    default:
        System.out.println("6번이 나왔습니다.");
}
```

코드 실행 흐름 제어

- switch 문
 - break가 없다면 다음 case가 연달아 실행

```
int time = (int)(Math.random()*4) + 8;
System.out.println("[현재시간: " + time + " 시]");

switch(time) {
    case 8:
        System.out.println("출근합니다.");
    case 9:
        System.out.println("회의를 합니다.");
    case 10:
        System.out.println("업무를 봅니다.");
    default:
        System.out.println("외근을 나갑니다.");
}
```

코드 실행 흐름 제어

- switch 문
 - break가 없다면 다음 case가 연달아 실행

```
char grade = 'B';

switch(grade) {
    case 'A':
    case 'a':
        System.out.println("우수 회원입니다.");
        break;
    case 'B':
    case 'b':
        System.out.println("일반 회원입니다.");
        break;
    default:
        System.out.println("손님입니다.");
}
```

코드 실행 흐름 제어

- switch 문
 - 화살표 연산자 (->) 사용 가능. 마지막에 break가 포함된 것과 같음

```
String grade = "B";

//Java 12 이전 문법
int score1 = 0;
switch(grade) {
    case "A":
        score1 = 100;
        break;
    case "B":
        int result = 100 - 20;
        score1 = result;
        break;
    default:
        score1 = 60;
}

System.out.println("score1: " + score1);
```

```
String grade = "B";
//Java 12 부터 가능
int score2 = switch(grade) {
    case "A" -> 100;
    case "B" -> {
        int result = 100 - 20;
        //Java 13부터 가능
        yield result;
    }
    default -> 60;
};

System.out.println("score2: " + score2);
```

코드 실행 흐름 제어

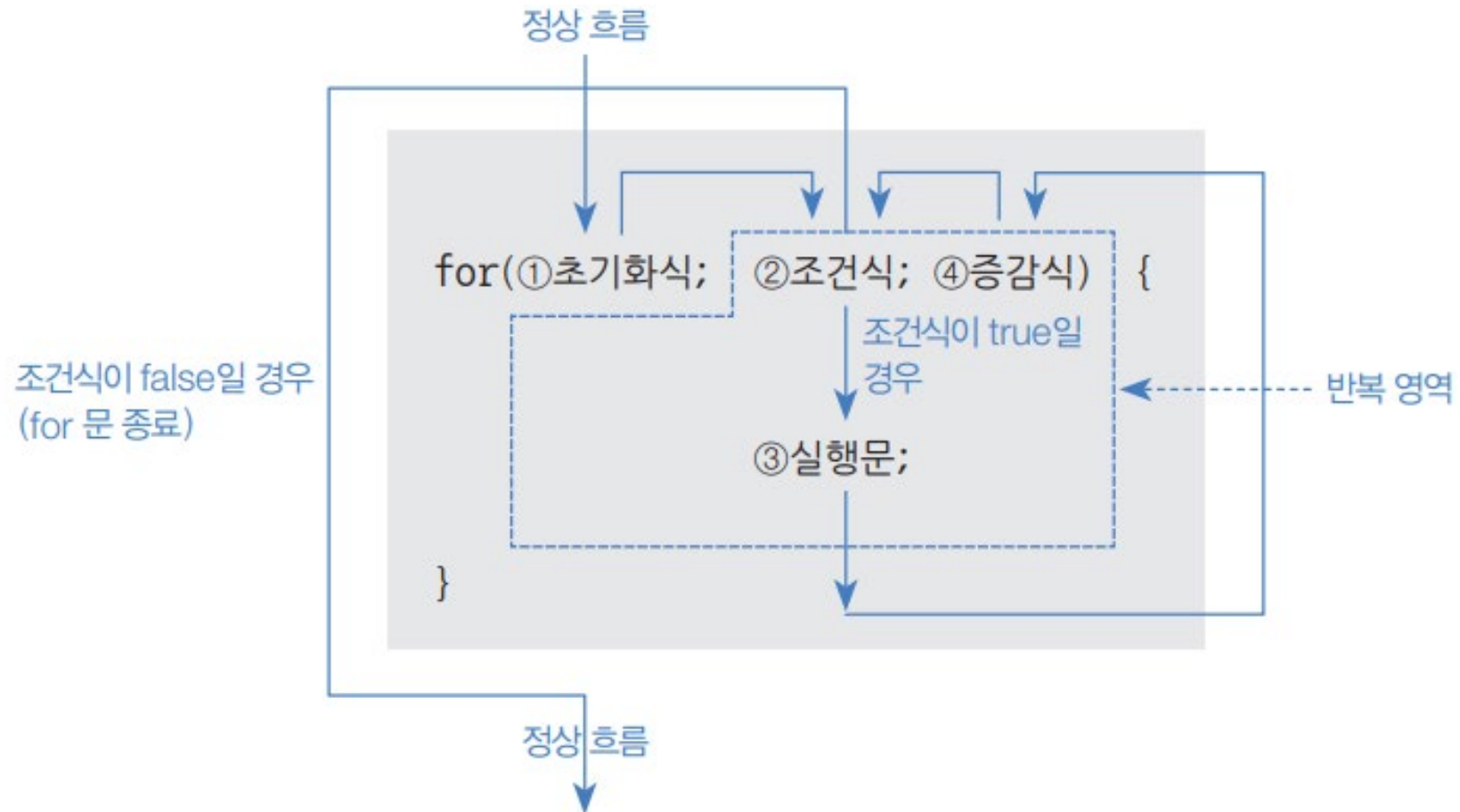
- for 문

- 실행문을 여러 번 반복 실행

- ①초기화식이 제일 먼저 실행 ②조건식을 평가해서 true이면 ③실행문을 실행시키고, false이면 for 문을 종료하고 블록을 건너뛴
 - ②조건식이 true가 되어 ③실행문을 모두 실행하면 ④증감식이 실행
 - 다시 ②조건식을 평가. 평가 결과가 다시 true이면 ③ → ④ → ②로 다시 진행하고, false이면 for 문이 끝남
 - 초기화식에서 부동 소수점을 쓰는 float 타입을 사용하지 않도록 주의

코드 실행 흐름 제어

- for 문



코드 실행 흐름 제어

- for 문

실수 프린트 하기

```
public static void main(String[] args) {  
    for(float x=0.1f; x<=1.0f; x+=0.1f) {  
        System.out.println(x);  
    }  
}
```

구구단 프린트 하기

```
for (int m=2; m<=9; m++) {  
    System.out.println("*** " + m + "단 ***");  
    for (int n=1; n<=9; n++) {  
        System.out.println(m + " x " + n + " = " + (m*n));  
    }  
}
```

배열을 for문에 사용하기

```
int[] intList = {1, 0, 3, 4, 5, 2};  
for (int i : intList) {  
    System.out.println(i);  
}
```

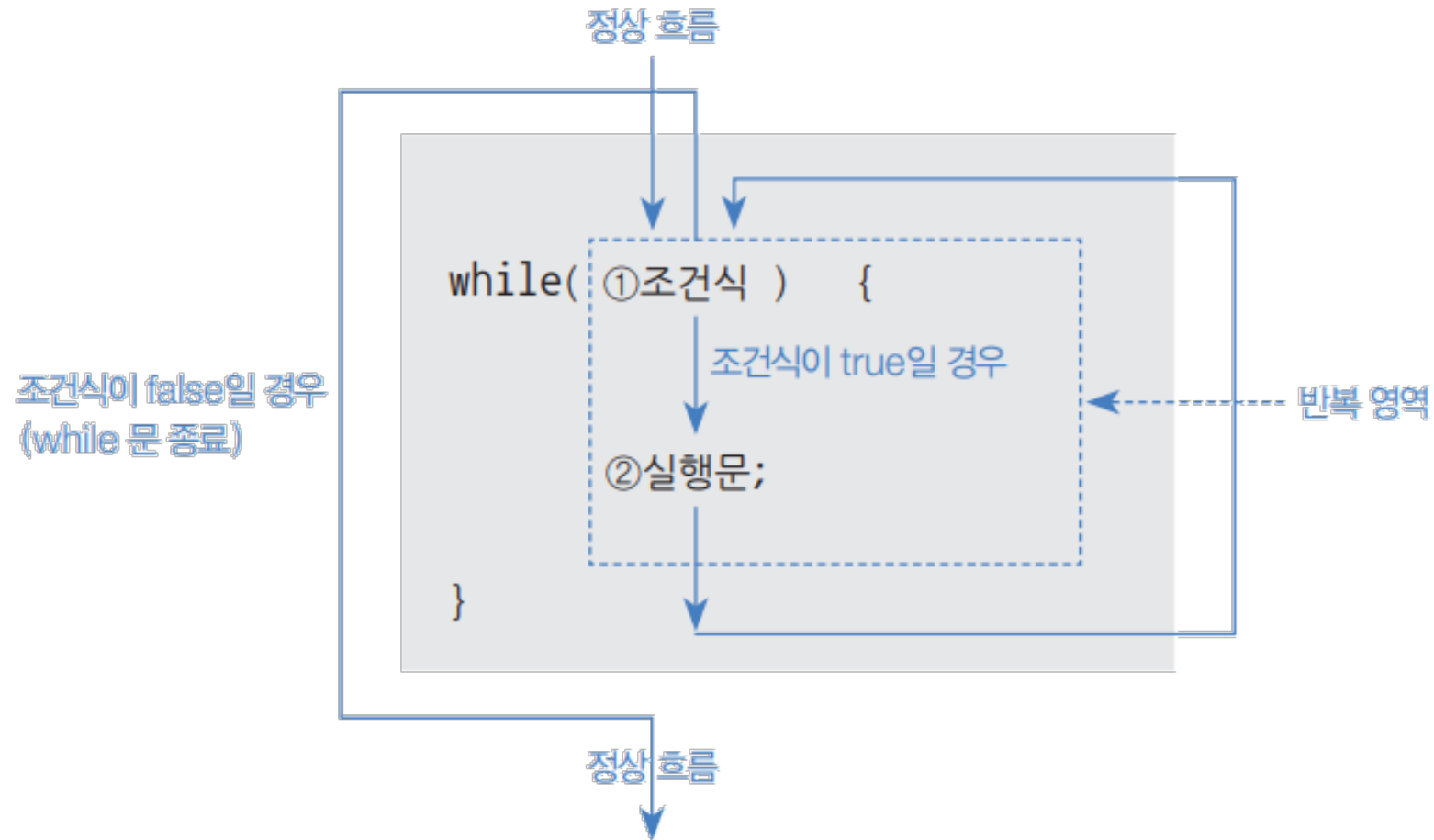
코드 실행 흐름 제어

- while 문

- 조건식이 true일 경우에 계속해서 반복하고, false가 되면 while 문을 종료
 - while 문이 처음 실행될 때 ①조건식을 평가. 평가 결과가 true이면 ②실행문을 실행하고 평가 결과가 false 이면 while 문 종료하고 블록을 건너 뛴
 - ②실행문이 모두 실행되면 조건식으로 되돌아가서 ①조건식을 다시 평가. 다시 조건식이 true라면 ② → ①로 진행하고, false라면 while 문을 종료
 - 조건식에 true를 사용하면 while(true) {...}가 되어서 무한 반복
 - 이 경우 while 문을 빠져나가기 위한 코드 필요

코드 실행 흐름 제어

- while 문



코드 실행 흐름 제어

- while 문

정수 프린트 하기

```
int i = 1;
while (i<=10) {
    System.out.print(i + " ");
    i++;
}
```

정수의 합 프린트 하기

```
int sum = 0;

int i = 1;

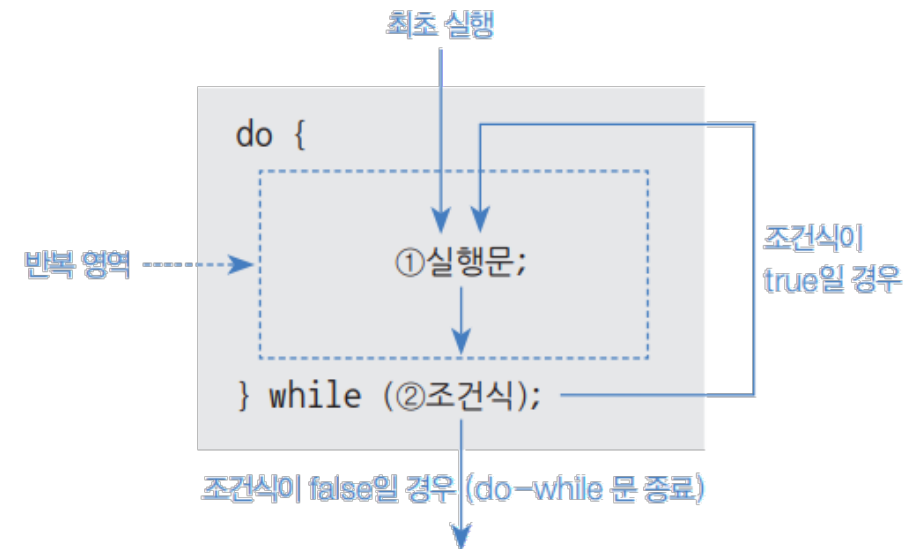
while(i<=100) {
    sum += i;
    i++;
}

System.out.println("1~" + (i-1) + " 합 : " + sum);
```

코드 실행 흐름 제어

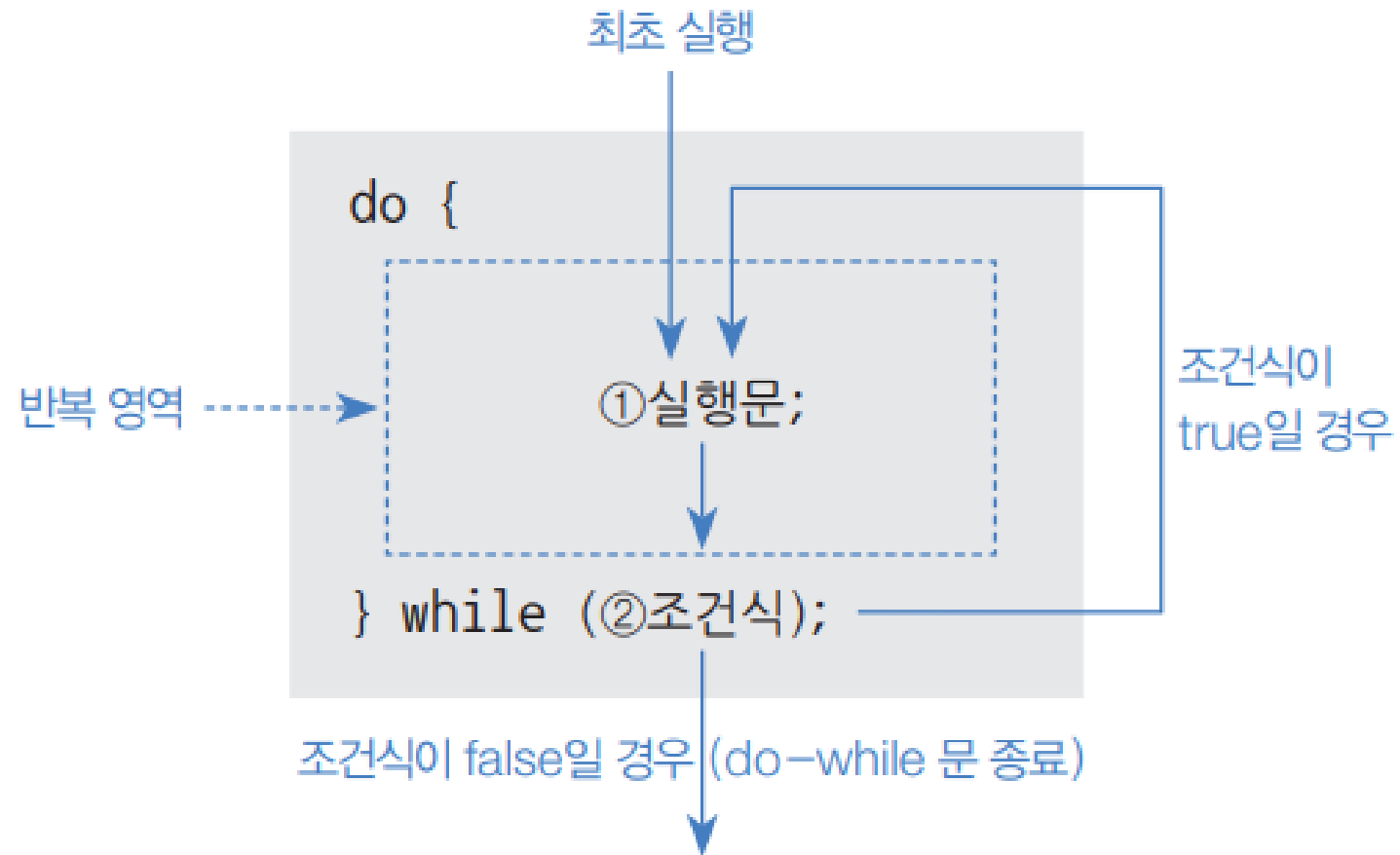
- do-while 문

- do 블록을 일단 실행시키고, 결과에 따라서 반복 실행을 계속할지 결정
 - do-while 문이 처음 실행될 때 ①실행문을 우선 실행한다. ①실행문이 모두 실행되면 ②조건식을 평가
 - 평가 결과가 true이면 ① → ②와 같이 반복 실행을 하고, 조건식의 결과가 false이면 do-while 문을 종료한다.
 - while() 뒤에 반드시 세미콜론(;)을 붙여야 함



코드 실행 흐름 제어

- do-while 문



코드 실행 흐름 제어

- do-while 문

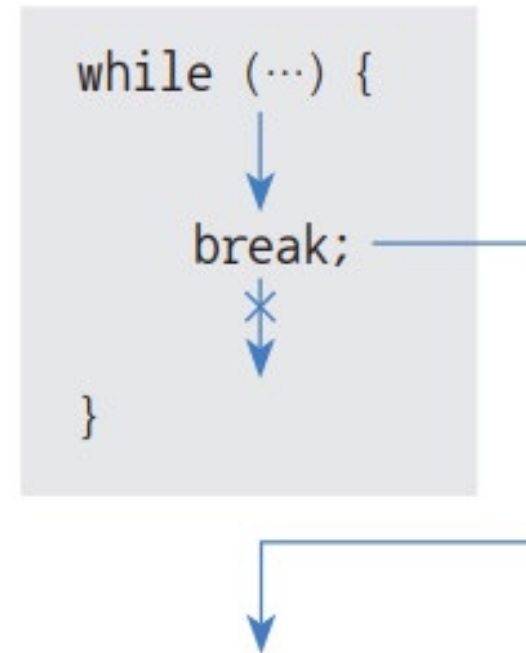
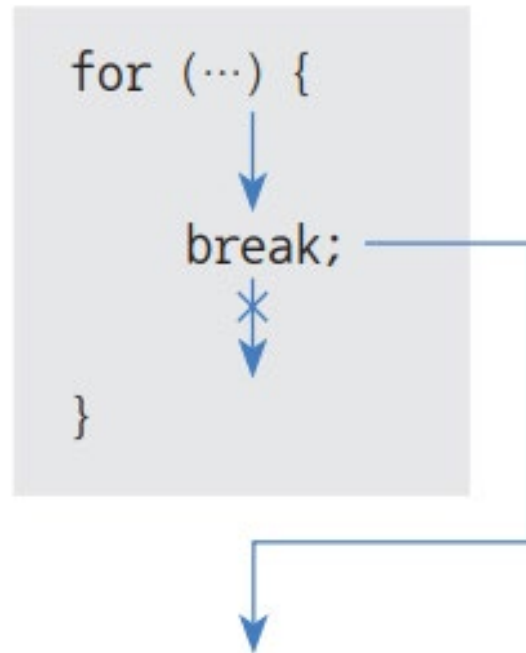
```
System.out.println("메시지를 입력하세요");  
System.out.println("프로그램을 종료하려면 q를 입력하세요.");  
  
Scanner scanner = new Scanner(System.in);  
String inputString;  
  
do {  
    System.out.print(">");  
    inputString = scanner.nextLine();  
    System.out.println(inputString);  
} while( ! inputString.equals("q") );  
  
System.out.println();  
System.out.println("프로그램 종료");
```


코드 실행 흐름 제어

- Scanner: 키보드 입력을 받는 클래스
 - `.nextLine()` : 엔터가 눌러질때까지 입력된 문자열을 받음
 - `.next()` : 공백 전까지 입력된 문자열을 받음
 - `.nextInt()` : int 자료형을 입력 받음

코드 실행 흐름 제어

- break 문
 - 반복문인 for 문, while 문, do-while 문을 실행 중지하거나 조건문인 switch 문을 종료할 때 사용
 - break 문은 대개 if 문과 같이 사용되어 조건식에 따라 for 문과 while 문을 종료



코드 실행 흐름 제어

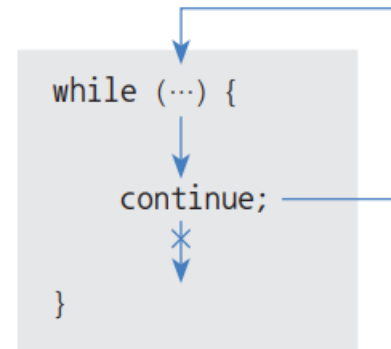
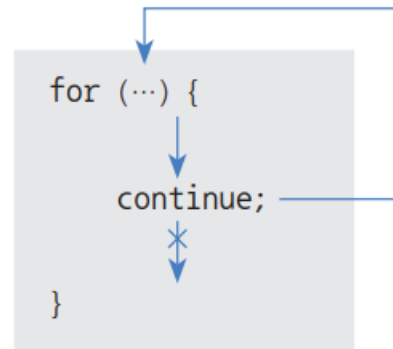
- break 문

6이 선택되면 종료

```
while(true) {  
    int num = (int)(Math.random()*6) + 1;  
    System.out.println(num);  
    if(num == 6) {  
        break;  
    }  
}  
System.out.println("프로그램 종료");
```

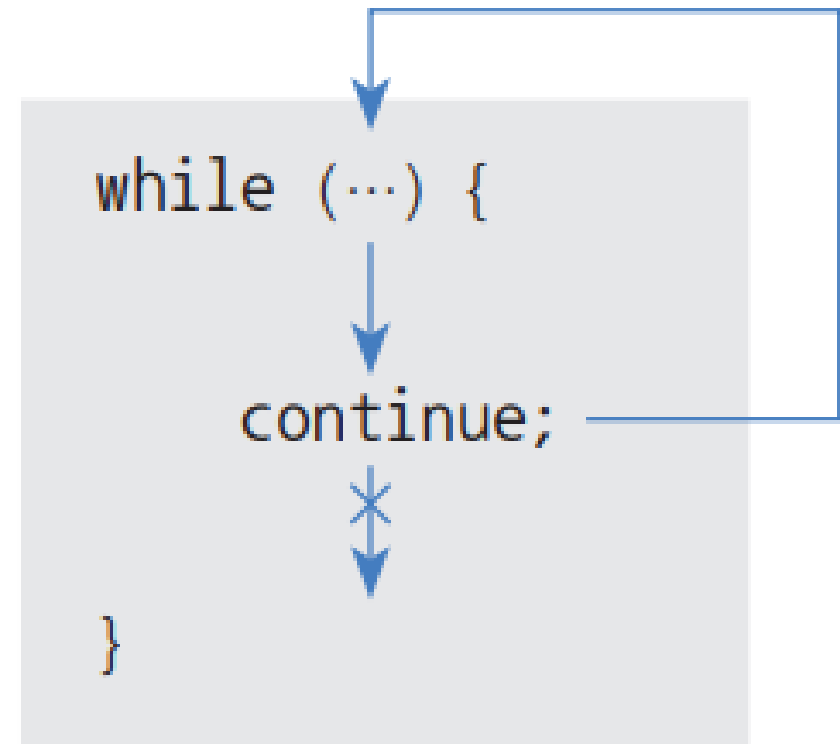
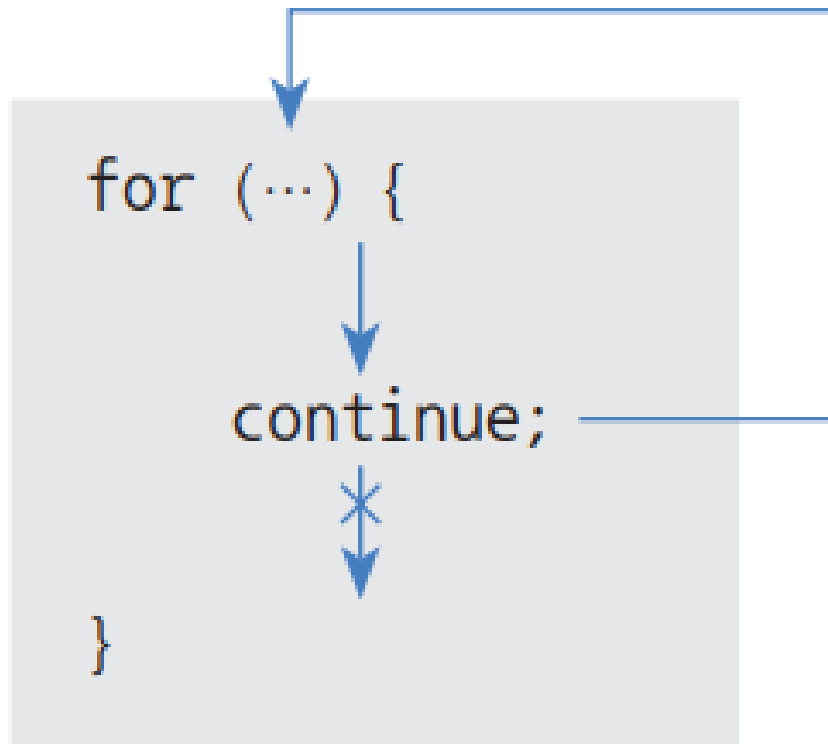
코드 실행 흐름 제어

- continue 문
 - 반복문인 for 문, while 문, do-while 문에서만 사용
 - 블록 내부에서 continue 문이 실행되면 for 문의 증감식 또는 while 문, do-while 문의 조건식으로 바로 이동
 - break 문과 달리 반복문을 종료하지 않고 계속 반복을 수행
 - 대개 if 문과 같이 사용되며, 특정 조건을 만족하는 경우에 continue 문을 실행해서 그 이후의 문장을 실행하지 않고 다음 반복으로 넘어감



코드 실행 흐름 제어

- continue 문



코드 실행 흐름 제어

- continue 문

홀수는 건너뛰면서 프린트

```
for(int i=1; i<=10; i++) {  
    if(i%2 != 0) {  
        continue;  
    }  
    System.out.print(i + " ");  
}
```

